BaseCallbackHandler

Used for hooking into the lifecycle events of a chain/agent/LLM run.

```
from langchain.callbacks.base import BaseCallbackHandler

class MyHandler(BaseCallbackHandler):
    def on_llm_start(self, serialized, prompts, **kwargs):
        print("LLM call started:", prompts)

    def on_llm_new_token(self, token, **kwargs):
        print(token, end="", flush=True)  # useful for streaming

    def on_llm_end(self, response, **kwargs):
        print("LLM call finished:", response)

    def on_llm_error(self, error, **kwargs):
        print("LLM errored:", error)

    def on_chain_start(self, serialized, inputs, **kwargs):
        ...

    def on_chain_end(self, outputs, **kwargs):
        ...

    def on_tool_start(self, serialized, input_str, **kwargs):
        ...

    def on_tool_end(self, output, **kwargs):
        ...

    def on_agent_action(self, action, **kwargs):
        ...
```

Use cases:

- **Streaming tokens to a frontend** (e.g., a websock or terminal)
- **Custom logging/observability** (this is essentially what LangSmith's tracer is, a callback that ship the data to the backend)
- **Cost/token tracking**
- **Debugging:** LangChain ships `StdOutCallbackHandler`
- **Guardrails:** intercepting or halting on certain outputs.

AsyncCallbackHandler

for cases that handler needs to do async work (e.g., writing to a database or making network calls without blocking)

One gotcha people run into: if you're using streaming, you generally need to pass `streaming=True` to the LLM *and* attach a handler that implements `on_llm_new_token`, otherwise tokens won't stream to your callback even though the API itself streams them.